

UNIDAD 6: EXCEPCIONES

ÍNDICE

1. INTRODUCCIÓN	1
2. ¿QUÉ ES UNA EXCEPCIÓN?	4
3. MANEJO DE EXCEPCIONES.....	8
1. ATRAPAR UNA EXCEPCIÓN.....	8
2. LANZAR UNA EXCEPCIÓN	12
3. TIPOS PROPIOS DE EXCEPCIONES.....	13
4. ERRORES FRECUENTES	16
5. EXCEPCIONES IOEXCEPTION CON CLASE SCANNER	20
6. EXCEPCIONES CON SPRING	21
@ControllerAdvice	21
Uso principal: Manejo global de excepciones.....	22
¿Cómo funciona internamente?.....	22
Componentes clave.....	22
Ejemplo completo (flujo real).....	23
Otros usos de @ControllerAdvice	23
Versiones especializadas (no son de este curso).....	24
Ventajas reales.....	24
Buenas prácticas	24



1. INTRODUCCIÓN

Existe una regla de oro en el mundo de la programación:

EN LOS PROGRAMAS, OCURREN ERRORES

Esto es sabido. Pero ¿qué sucede realmente después de que ha ocurrido el error? ¿Cómo se maneja el error? ¿Quién lo maneja? ¿Puede recuperarlo el programa?

El lenguaje Java utiliza excepciones para proporcionar capacidades de manejo de errores. En esta lección aprenderás qué es una excepción, cómo lanzar y capturar excepciones, qué hacer con una excepción una vez capturada, y cómo hacer un mejor uso de las excepciones heredadas de las clases proporcionadas por el entorno de desarrollo de Java.

Por desgracia, ya nos hemos encontrado con excepciones a lo largo de estos meses como, por ejemplo:

- **ArrayIndexOutOfBoundsException** cuando se intenta acceder a una posición de un array que está fuera de los límites de este.
- **NullPointerException** cada vez que se utiliza una referencia null en donde se espera un objeto y este todavía no ha sido instanciado.
- **NumberFormatException** cuando se lee un número que no tiene el formato esperado.
- ...

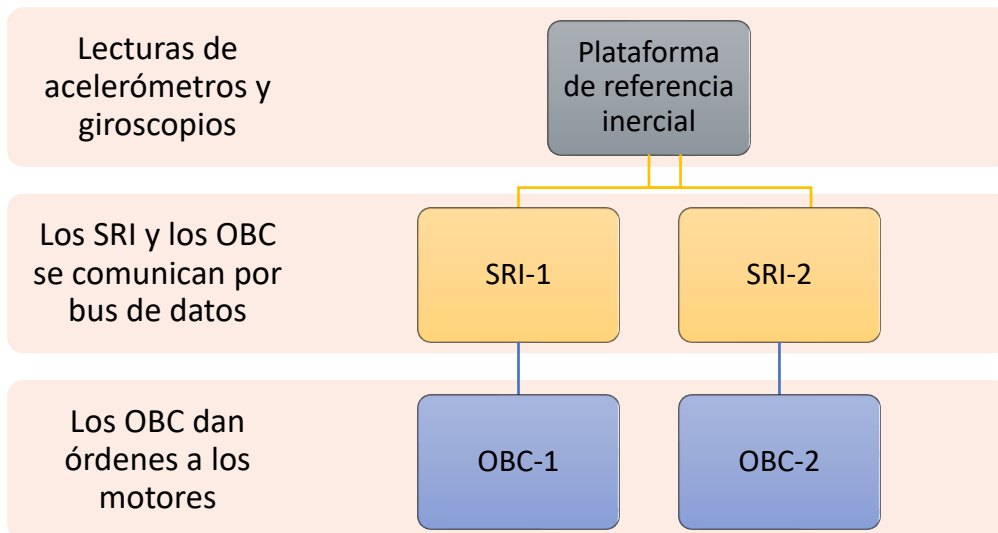
¿Es importante tratar las excepciones? Veámoslo con un ejemplo real:

https://www.youtube.com/watch?v=PK_yguLapgA

Caso del Ariane 5: Vuelo 501, 4 de junio de 1996. Coste: 500 millones de dólares.



SISTEMA DE CONTROL DE VUELO



El informe del equipo de investigación fue el siguiente:

Aproximadamente a los 39 segundos, el Ariane 5 comenzó a desintegrarse debido a los esfuerzos aerodinámicos excesivos ocasionados por un ángulo de ataque inapropiado. Esto condujo a la separación de los propulsores auxiliares respecto del cuerpo principal de la nave, por lo que se disparó el mecanismo de **AUTODESTRUCCIÓN** del cohete.

El ángulo de ataque resultó ser consecuencia de una completa desalineación entre las toberas de los propulsores auxiliares y la del motor principal. La desalineación de las toberas fue ordenada por el ordenador de abordo a partir de información transmitida por el sistema de referencia inercial (SRI-2). Parte de estos datos no contenían información de vuelo sino un patrón del SRI, el cual fue interpretado como información de vuelo.

La razón por la cual el SRI-2 no envió datos reales es que la unidad se había declarado en falla por una **excepción de software**.

El ordenador de a bordo, no pudo acceder al SRI-1 porque este se había desactivado algunos milisegundos antes por la misma razón que el SRI-2.

La excepción interna en el software del SRI fue ocasionada durante la ejecución de una instrucción de conversión de un valor en formato punto flotante (Double) de 64 bits a entero con signo de 16 bits, en circunstancias que el valor excedía la capacidad del tipo entero. La operación de conversión no estaba protegida aun cuando asignaciones de variables similares sí lo estaban...

La operación inválida generaba correctamente una excepción, sin embargo, el mecanismo de gestión de excepciones no fue diseñado para responder adecuadamente.

El mecanismo de tolerancia a fallas del Ariane 5 estaba concebido para tratar fallas aleatorias (por ejemplo, mal funcionamiento del hardware de uno de los ordenadores) pero no errores de diseño (ambas SRI experimentaron el mismo comportamiento al momento de producirse la falla).

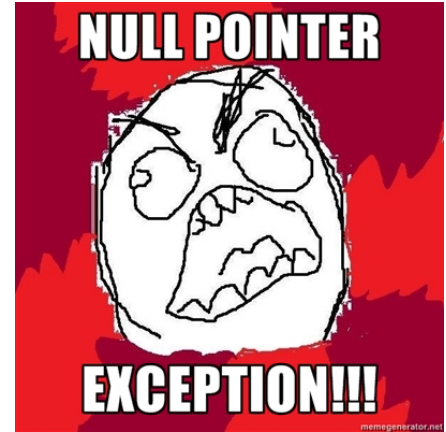
El equipo de investigación sugirió cambiar el **enfoque de desarrollo utilizado hasta entonces**:

“El software se considera correcto hasta que se demuestra que falla” por el enfoque “se asume que el software es defectuoso hasta que, tras aplicar métodos aceptados como buenas prácticas, se logre demostrar que está correcto”.

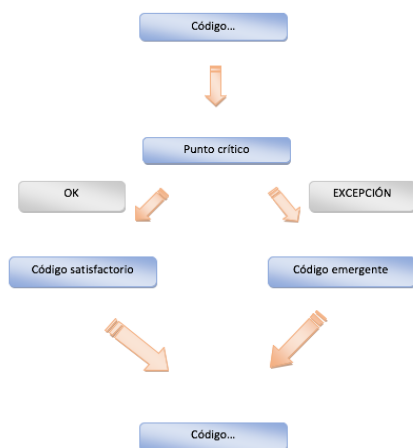
2. ¿QUÉ ES UNA EXCEPCIÓN?

Una **excepción** es la indicación de un problema que ocurre durante la **ejecución de un programa** y que, si no se gestiona adecuadamente, provoca la interrupción abrupta de la aplicación.

El nombre “excepción” implica que el problema ocurre con poca frecuencia; si la “regla” es que una instrucción generalmente se ejecuta en forma correcta, entonces la “excepción a la regla” es cuando ocurre un problema. El manejo de excepciones permite crear aplicaciones que puedan resolver (o manejar) las excepciones. En muchos casos, el manejo de una excepción permite que el programa continúe su ejecución como si no se hubiera encontrado el problema. Un problema más grave podría evitar que un programa continuara su ejecución normal, en vez de requerir al programa que notifique al usuario sobre el problema antes de terminar de una manera controlada. Los aspectos que estudiaremos en esta unidad, permiten a los programadores escribir **programas tolerantes a fallas y robustos** (es decir, programas que traten con los problemas que puedan surgir sin dejar de ejecutarse).



Todo programador debe, por tanto, contemplar el caso ideal pero también las contingencias que puedan ocurrir y cómo corregir el rumbo.



Una excepción en Java es **cualquier objeto** de clase `java.lang.Exception` o subclase de la misma (si el programador desea crear **sus propias excepciones** deberá entonces crear subclases de la clase mencionada).

```

public class MiExcepcion extends Exception {

    public MiExcepcion ( ) {
        super ("Esta es una excepción provocada");
    }
}
  
```

¿Qué ocurre con una excepción?

- Cuando se genera una excepción el intérprete crea un objeto para representar la excepción con información sobre la misma.
- Envía (lanza) el objeto (la excepción) al método que ha provocado la excepción.
- Si el método no captura la excepción debe lanzarla hasta que alguno la capture y trate. Si al final, ningún método la captura, entonces el intérprete la captura y realiza las acciones pertinentes (detención del programa y avisos o mensajes por pantalla).

Java tiene un sistema para que el programador defina la captura y gestión de las excepciones.

Cuando un error ocurre dentro de un método Java, el método crea un objeto 'Exception' y lo maneja fuera, en el sistema de ejecución. Este objeto contiene información sobre la excepción, incluyendo su tipo y el estado del programa cuando ocurrió el error. El sistema de ejecución es el responsable de buscar algún código para manejar el error. En terminología java, crear un objeto Exception y manejarlo por el sistema de ejecución se llama **lanzar una excepción**.

Después de que un método lance una excepción, el sistema de ejecución entra en acción para buscar el manejador de la excepción. El conjunto de "algunos" métodos posibles para manejar la excepción es el conjunto de métodos de la pila de llamadas del método donde ocurrió el error. **El sistema de ejecución busca hacia atrás en la pila de llamadas, empezando por el método en el que ocurrió el error, hasta que encuentra un método que contiene el "manejador de excepción" adecuado (catch).**

Un manejador de excepción es considerado adecuado si el tipo de la excepción lanzada es el mismo que el de la excepción manejada por el manejador. Así la excepción sube sobre la pila de llamadas hasta que encuentra el manejador apropiado y una de las llamadas a métodos maneja la excepción, se dice que el manejador de excepción elegido **captura la excepción**.

Si el sistema de ejecución busca exhaustivamente por todos los métodos de la pila de llamadas sin encontrar el manejador de excepción adecuado, el sistema de ejecución finaliza (y consecuentemente y el programa Java también).

Mediante el uso de excepciones para manejar errores, los programas Java tienen las siguientes ventajas:

- ✓ Separar el manejo de errores del código "Normal".
- ✓ Permite propagar los errores sobre la Pila de Llamadas.
- ✓ Agrupar los tipos de errores y la diferenciación de éstos mediante la jerarquía de clases.

Si no gestionas excepciones te pueden pasar cosas como esta:

```
public class EjemExcepciones {
    public static void main(String[] args) {
        int a[ ] = new int[3];
        for ( inti = 0; i<= 3; i++){
            a [i] = i;
        }
        metodo( a );
    }

    static void método (int b[ ] ) {
        b [2] = 1 / b[0];
    }
}
```

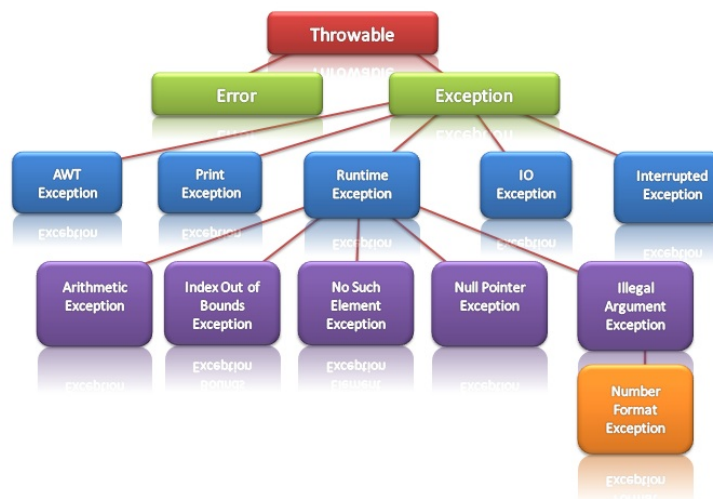
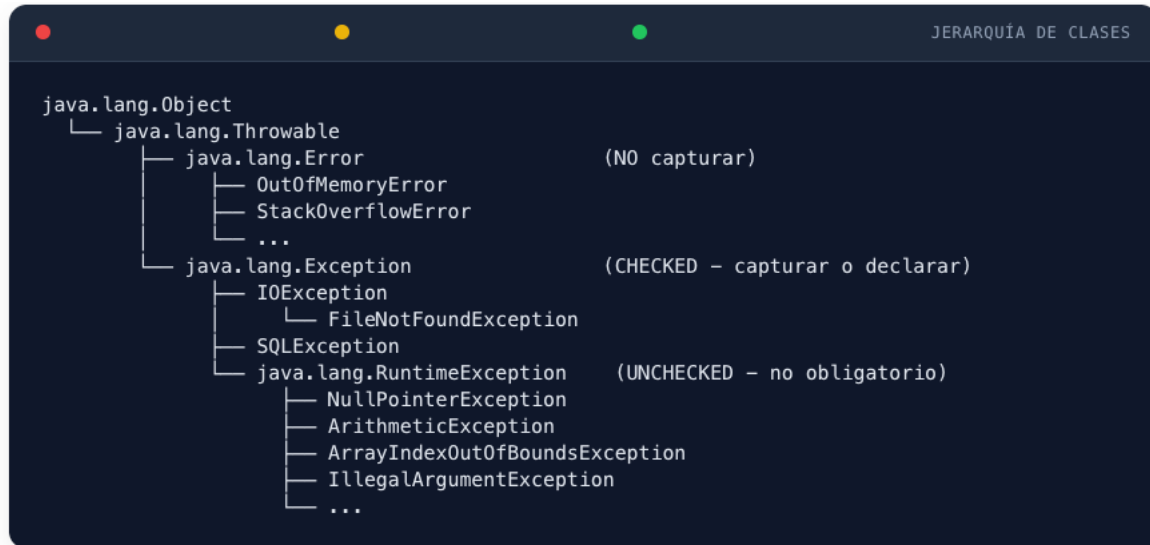
java.lang.ArrayIndexOutOfBoundsException: 3at
EjemExcepciones.EjemExcepciones.main(EjemExcepciones.java:7)
Exception in thread "main"

El intérprete lanza una excepción del tipo (clase) ArrayIndexOutOfBoundsException, es decir, el error se ha producido por intentar acceder a una posición externa al array. Además el intérprete nos informa de la posición o índice incorrecto dentro de la matriz (3), así como del método que ha provocado la excepción.

java.lang.ArithmeticException: / by zero
atEjemExcepciones.EjemExcepciones.fun(EjemExcepciones.java:11)
atEjemExcepciones.EjemExcepciones.main(EjemExcepciones.java:11)

El intérprete lanza una excepción del tipo (clase) ArithmeticException, es decir, el error se produce al dividir uno por cero. El intérprete nos informa además del método que produjo la excepción

Jerarquía de excepciones en Java



En la figura muestra una pequeña porción de la jerarquía de herencia para la clase **Throwable** (una subclase de **Object**), que es la superclase de la clase **Exception**. Sólo pueden usarse objetos **Throwable** con el mecanismo para manejar excepciones. La clase **Throwable** tiene dos subclases: **Exception** y **Error**.

La clase **Exception** y sus subclases (por ejemplo, **RuntimeException** del paquete `java.lang` e **IOException**, del paquete `java.io`) representan situaciones excepcionales que pueden ocurrir en un programa en Java, y que pueden ser atrapadas por la aplicación.

La clase **Error** y sus subclases (por ejemplo, **OutOfMemoryError**) representan situaciones anormales que podrían ocurrir en la JVM. Los errores tipo **Error** ocurren con poca frecuencia y no deben ser atrapados por las aplicaciones; por lo general, no es posible que las aplicaciones se recuperen de los errores tipo **Error**.

[Nota: la jerarquía de excepciones de Java contiene cientos de clases. En la API de Java se pueden encontrar información acerca de las clases de excepciones de Java]

Todas las excepciones y errores en Java son subclases de `java.lang.Throwable`. La jerarquía se divide en dos ramas principales:

CLASE	TIPO	DESCRIPCIÓN	EJEMPLOS
<code>Error</code>	Errores graves	Problemas graves del entorno de ejecución que no deben capturarse.	<code>OutOfMemoryError</code> , <code>StackOverflowError</code>
<code>Exception</code>	Excepciones checked	Errores recuperables. El compilador obliga a manejarlas.	<code>IOException</code> , <code>SQLException</code>
<code>RuntimeException</code>	Excepciones unchecked	Errores de programación. No requieren manejo obligatorio.	<code>NullPointerException</code> , <code>ArithmeticException</code>

Excepciones checked vs unchecked

Java clasifica a las excepciones en dos categorías: **excepciones verificadas (o comprobadas o checked)** y **excepciones no verificadas (o no comprobadas o unchecked)**. Esta distinción es importante, ya que el compilador de Java implementa un **requerimiento de atrapar o declarar** para las excepciones verificadas.

El tipo de una excepción determina si es verificada o no verificada.

Todos los tipos de excepciones que son subclases directas o indirectas de la clase **`RuntimeException`** (paquete `java.lang`) son excepciones **no verificadas**. Esto incluye a las excepciones que ya hemos visto, como las excepciones `ArrayIndexOutOfBoundsException` y `ArithmeticException`.

Todas las clases que **heredan de la clase `Exception` pero no de la clase `RuntimeException`** se consideran como excepciones **verificadas**; y las que heredan de la clase `Error` se consideran como no verificadas. El compilador *verifica* cada una de las llamadas a un método, junto con su declaración, para determinar si el método lanza excepciones verificadas. De ser así, el compilador asegura que la excepción verificada sea atrapada o declarada en una cláusula `throws`.

Mis neuronas eligiendo el nombre de una variable



Aah pero cuando eligen el nombre de una variable para iterar



CARACTERÍSTICA	CHECKED	UNCHECKED
Hereda de	<code>Exception</code> (no <code>RuntimeException</code>)	<code>RuntimeException</code>
Verificación	En tiempo de compilación	En tiempo de ejecución
Obligación	Hay que capturarla o declararla con <code>throws</code>	No hay obligación
Causa típica	Condiciones externas: fichero no encontrado, conexión caída	Errores de programación: índice fuera de rango
Ejemplo	<code>IOException</code> , <code>SQLException</code>	<code>NullPointerException</code> , <code>ArithmeticException</code>

La cláusula throws especifica las excepciones que lanza un método. Dichas excepciones no se atrapan en el cuerpo del método.

- Para satisfacer la parte relacionada con atrapar del requerimiento de atrapar o declarar, el código que genera la excepción debe envolverse en un bloque try, y debe proporcionar un manejador catch para el tipo de excepción verificada (o uno de los tipos de su superclase).
- Para satisfacer la parte relacionada con declarar del requerimiento de atrapar o declarar, el método que contiene el código que genera la excepción debe proporcionar una cláusula throws que contenga el tipo de excepción verificada, después de su lista de parámetros y antes de su cuerpo.
- Si el requerimiento de atrapar o declarar no se satisface, el compilador emitirá un mensaje de error, indicando que la excepción debe ser atrapada o declarada. Esto obliga a los programadores a **pensar** acerca de los problemas que pueden ocurrir cuando se hace una llamada a un método que lanza excepciones verificadas. Las clases de excepciones se definen para verificarse cuando se consideran lo bastante importantes como para atraparlas o declararlas.

$$0^0 = x$$

$$\sqrt[n]{0^0} = \sqrt[n]{x}$$

$$0 = \sqrt[n]{x} \quad | : \sqrt[n]{x}$$

$$0 = \frac{\sqrt[n]{x}}{\sqrt[n]{x}}$$

$$0 = 1$$

QUANTUM, OFFICER?



A diferencia de las excepciones verificadas, el compilador de Java no verifica el código para determinar si una excepción no verificada es atrapada o declarada. Por lo general, las excepciones no verificadas se pueden evitar mediante una codificación apropiada. Por ejemplo, la excepción ArithmeticException no verificada que lanza el método cociente puede evitarse si el método se asegura que el denominador no sea cero antes de tratar de realizar la división. No es obligatorio que se listen las excepciones no verificadas en la cláusula throws de un método; aun si se listan, no es obligatorio que una aplicación atrape dichas excepciones.

3. MANEJO DE EXCEPCIONES

1. ATRAPAR UNA EXCEPCIÓN

Cuando un método se encuentra con una anomalía que no puede resolver, lo lógico es que lance (throw) una excepción, esperando que quien lo llamó directa o indirectamente la atrape (catch) y maneje la anomalía. Incluso él mismo podría atrapar y manipular dicha excepción. Si la excepción no se atrapa, el programa finalizará automáticamente.

Veamos un ejemplo concreto:

a) Primero sin capturar la excepción



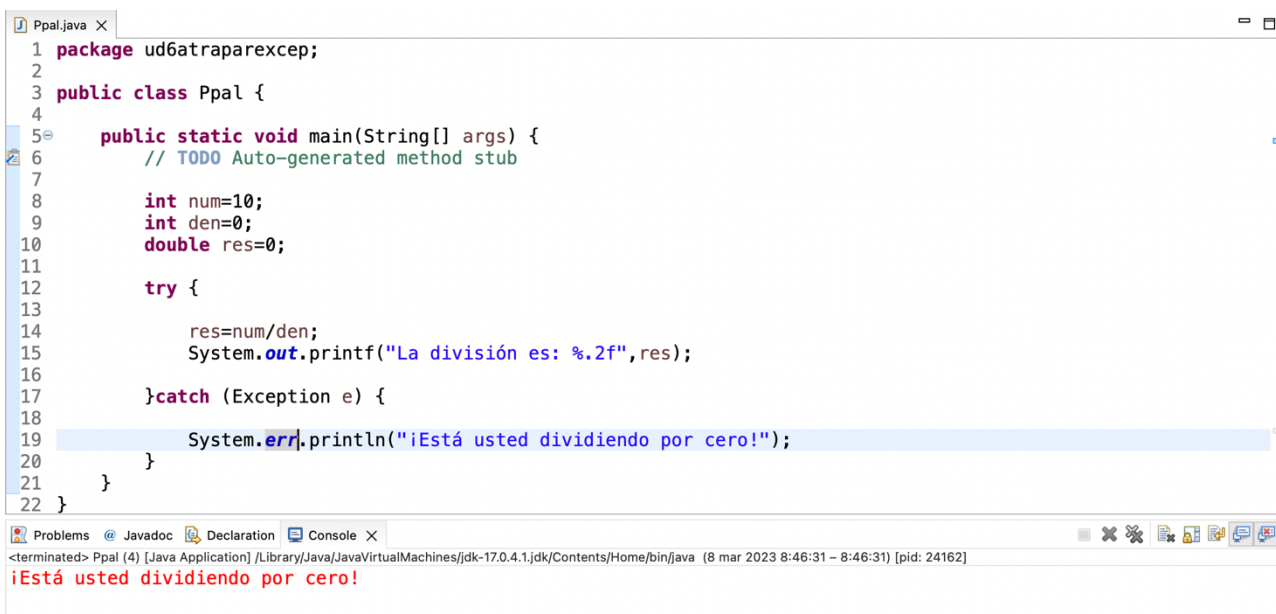
```

1 package ud6atraparexcep;
2
3 public class Ppal {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8         int num=10;
9         int den=0;
10        double res=0;
11
12        res=num/den;
13
14        System.out.printf("La división es: %.2f"+res);
15
16    }
17 }
18
19 }
20

```

Exception in thread "main" java.lang.ArithmeticException: / by zero
at Pruebas2223/ud6atraparexcep.Ppal.main(Ppal.java:12)

b) Atrapando la excepción



```

1 package ud6atraparexcep;
2
3 public class Ppal {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8         int num=10;
9         int den=0;
10        double res=0;
11
12        try {
13
14            res=num/den;
15            System.out.printf("La división es: %.2f",res);
16
17        }catch (Exception e) {
18
19            System.err.println("¡Está usted dividiendo por cero!");
20        }
21    }
22 }

```

¡Está usted dividiendo por cero!

1) Bloque try:

En la parte try debe escribirse la parte de código que es susceptible de producir una excepción, es decir, parte del código donde se puede dar un error, no solo la línea en cuestión, se suele poner todo el código relacionado para no tener que escribir muchos bloques try, cosa que hace menos fácil de leer el código.

Detrás del bloque try siempre debe ir un bloque catch o un bloque finally.

En este bloque, cuando se genera una excepción, automáticamente el flujo del programa pasa a la llave final del try y comienza a buscar que bloque catch puede atrapar la excepción.

2) Bloque catch:

Cuando se genera una excepción, la ejecución del programa pasa al bloque catch cuyo parámetro coincida con el tipo de excepción generada.

Obligatoriamente deben ir con el bloque try. Aquí va el trozo de código que debe atrapar la excepción lanzada en el bloque try.

Después de la palabra catch se abren paréntesis y dentro de estos se debe escribir el tipo de excepción que atrapa ese bloque.

OJO: Se pueden escribir varios bloques catch uno detrás de otro, tantos como tipos de excepciones queramos atrapar, pero siempre se debe seguir un orden determinado, **el primer catch debe ser el dedicado a la excepción MÁS CONCRETA** (más abajo en la jerarquía de excepciones) y así hacia abajo hasta llegar a la última que debe ser el tipo de excepción más genérica, por ejemplo, la última debería ser la Exception. Si, por ejemplo, la más genérica fuera la primera (más arriba en el código) este catch atraparía todas las excepciones.

Dentro de cada bloque irá el código o mensaje que se mostrará cuando tenga lugar ese tipo de excepción o se lanzará una nueva excepción.

Cada catch solo puede atrapar un tipo de excepción, no se pueden escribir más de un tipo dentro de los paréntesis.

Tampoco es posible escribir código entre los bloques try-catch.

3) Bloque Finally:

Este bloque se ejecuta SIEMPRE, se genere la excepción o no, con lo que este bloque se utiliza para realizar alguna acción obligatoria, por ejemplo, cerrar un fichero o liberar recursos. **Va siempre al final**, es decir, detrás de los bloques catch.

En resumen:

BLOQUE	OBLIGATORIO	SE EJECUTA CUANDO...
<code>try</code>	Sí	Siempre. Contiene el código que puede lanzar excepciones.
<code>catch</code>	Al menos uno (o un <code>finally</code>)	Solo si se lanza una excepción del tipo especificado.
<code>finally</code>	No	Siempre, haya o no excepción. Ideal para liberar recursos.

Ejemplo:

```
try {
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error aritmético: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Error genérico: " + e.getMessage());
} finally {
    System.out.println("Limpieza completada");
}
// Salida:
// Error aritmético: / by zero
// Limpieza completada
```

***NOTA:** Desde la versión Java 7, se pueden usar los bloques multicatch y la gestión automática de recursos usando try-with-resources. Puedes ver lo fundamental en el siguiente ejemplo pero se deja esto para que lo investigues por tu cuenta.*

Multicatch

```
try {
    String texto = null;
    int longitud = texto.length(); // NullPointerException
} catch (NullPointerException | ArrayIndexOutOfBoundsException e) {
    System.out.println("Error: " + e.getClass().getSimpleName() + " - " + e.getMessage());
}
// Salida: Error: NullPointerException - null
```

Try-with-resources (cierra automáticamente los recursos que implementan `AutoCloseable` al finalizar el bloque)

En el siguiente ejemplo, se cierra el buffer `datos.txt` al terminar la ejecución del `while` sin necesidad de añadir un bloque `finally`.

Es buena práctica utilizar este `try-with-resources` siempre que se trabaje con recursos que requieran cierre, por ejemplo, `FileReader`, `Connection`, `Scanner`).

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerArchivo {
    public static void main(String[] args) {
        try (BufferedReader lector = new BufferedReader(new FileReader("datos.txt"))) {
            String linea;
            while ((linea = lector.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer archivo: " + e.getMessage());
        }
        // No es necesario lector.close() en un finally
    }
}
```

2. LANZAR UNA EXCEPCIÓN

Aquellos métodos que puedan lanzar una excepción deben declararla. Ya hemos visto cómo se atrapan.

Para lanzarla al método que lo llamó basta con añadir la palabra “throws” después del paréntesis donde ponemos los parámetros del método y el tipo de excepción lanzada. Si el método lanza más de una se pueden añadir separadas por comas antes del cuerpo del método.

Dichas excepciones pueden lanzarse mediante instrucciones en el cuerpo del método, o mediante métodos que se llamen desde el cuerpo. Un método puede lanzar excepciones de las clases que se listen en su cláusula throws, o en la de sus subclases.

Para lanzar la excepción dentro del código se usará la palabra throw seguida de una referencia al objeto excepción.

ASPECTO	THROW	THROWS
Ubicación	Dentro del cuerpo de un método	En la firma del método
Función	Lanza una excepción explícitamente	Declara que el método puede propagar una excepción
Sintaxis	<code>throw new Exception("msg");</code>	<code>void metodo() throws Exception { }</code>
Cantidad	Lanza una sola excepción cada vez	Puede declarar varias, separadas por comas

Ejemplo de validación con throw y throws:

```
public class Validador {
    public static void validarEdad(int edad) throws IllegalArgumentException {
        if (edad < 0 || edad > 150) {
            throw new IllegalArgumentException(
                "Edad no válida: " + edad + ". Debe estar entre 0 y 150.");
        }
        System.out.println("Edad " + edad + " validada correctamente.");
    }

    public static void main(String[] args) {
        try {
            validarEdad(25);    // OK
            validarEdad(-5);    // Lanza excepción
            validarEdad(30);    // No se ejecuta
        } catch (IllegalArgumentException e) {
            System.out.println("Error de validación: " + e.getMessage());
        }
    }
}

// Salida:
// Edad 25 validada correctamente.
// Error de validación: Edad no válida: -5. Debe estar entre 0 y 150.
```

3. TIPOS PROPIOS DE EXCEPCIONES

Una de las cosas más importantes es conocer los distintos tipos de excepciones que pueden ocurrir. Java proporciona muchas clases en el API para la gestión de excepciones, pero también, el programador puede crear sus propias excepciones, sobre esto, se puede leer más en el apartado 13.11 del libro Deitel.

El programador puede crear sus propias excepciones y utilizarlas para indicar errores, pueden extender de Exception (llamadas comprobadas) o de RuntimeException (llamadas No Comprobadas).

```
public static class MiExcepción extends Exception {}
```

Si queremos asociar mensajes con información adicional a una excepción nuestra, debemos escribir los constructores:

```
public static class MiExcepción extends Exception {

    // constructor sin argumentos

    public MiExcepción() {
        super();
    }

    // constructor con un string como argumento

    public MiExcepción(String infoAdicional) {
        super(infoAdicional);
    }
}
```

```
}
```

```
}
```

Un método donde se lanza una excepción comprobada, deberá:

- Tratarla con un bloque try-catch
- O declarar que la lanza con una cláusula throws

Sintaxis de la cláusula **throws**:

```
public tipo nombreMétodo(parámetros) throws ClaseExcepción1, ClaseExcepción2 {
    declaraciones;
    instrucciones; // lanzan las excepciones
}
```

Cláusula throws en métodos anidados

```
public void metodo3() throws MiExcepción {
    ...
    throw new MiExcepción();
}
```

Lanza la excepción, por eso lo indica

```
public void metodo2() throws MiExcepción {
    ...
    metodo3();
}
```

No trata la excepción, por eso indica que puede lanzarla

```
public void metodo1() {
    ...
    try {
        metodo2();
    } catch (MiExcepción e) {
    }
    ...
}
```

Trata la excepción, por eso no tiene la cláusula throws

Ejemplo:

```

// Excepción checked personalizada
class SaldoInsuficienteException extends Exception {
    private double saldoActual;
    private double cantidadSolicitada;

    public SaldoInsuficienteException(double saldo, double cantidad) {
        super("Saldo insuficiente. Disponible: " + saldo + " €, solicitado: " + cantidad + " €");
        this.saldoActual = saldo;
        this.cantidadSolicitada = cantidad;
    }

    public double getSaldoActual() { return saldoActual; }
    public double getCantidadSolicitada() { return cantidadSolicitada; }
}

class CuentaBancaria {
    private String titular;
    private double saldo;

    public CuentaBancaria(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public void retirar(double cantidad) throws SaldoInsuficienteException {
        if (cantidad > saldo) {
            throw new SaldoInsuficienteException(saldo, cantidad);
        }
        saldo -= cantidad;
        System.out.println("Retirada de " + cantidad + " €. Saldo: " + saldo + " €");
    }
}

public class EjemploExcepcionPersonalizada {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Ana García", 1000.0);
        try {
            cuenta.retirar(300.0);    // OK
            cuenta.retirar(900.0);    // Lanza SaldoInsuficienteException
        } catch (SaldoInsuficienteException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Faltan: " + (e.getCantidadSolicitada() - e.getSaldoActual()) + " €");
        }
    }
}

// Salida:
// Retirada de 300.0 €. Saldo: 700.0 €
// Error: Saldo insuficiente. Disponible: 700.0 €, solicitado: 900.0 €
// Faltan: 200.0 €
```

Buenas prácticas en el manejo de excepciones

PRÁCTICA	DESCRIPCIÓN
Capturar excepciones específicas	Usar <code>catch (IOException e)</code> en lugar de <code>catch (Exception e)</code> .
No silenciar excepciones	Un <code>catch</code> vacío es un antipatrón grave. Al mínimo, registrar el error.
Usar try-with-resources	Para ficheros, conexiones, streams. Evita fugas de recursos.
Mensajes descriptivos	Incluir contexto útil: <code>throw new IOException("No se pudo abrir: " + ruta)</code> .
No usar excepciones para control de flujo	Las excepciones son para situaciones excepcionales, no para lógica habitual.
Documentar con Javadoc	Documentar con <code>@throws</code> en el Javadoc del método.
Preferir unchecked para errores de programación	Usar <code>IllegalArgumentException</code> , <code>IllegalStateException</code> , etc.

Antipatrón vs buena práctica

● ● ● JAVA - ✗ ANTIPATRÓN: CATCH VACÍO <pre>try { conexion.ejecutarConsulta(sql); } catch (SQLException e) { // ✗ NUNCA: el error se pierde completamente }</pre>	● ● ● JAVA - ✔ BUENA PRÁCTICA <pre>try { conexion.ejecutarConsulta(sql); } catch (SQLException e) { System.err.println("Error en consulta: " + sql); e.printStackTrace(); throw new PersistenciaException("Fallo en consulta", e); }</pre>
---	--

4. ERRORES FRECUENTES

Error 1: Catch vacío que silencia el error

```
try {
    int resultado = Integer.parseInt(texto);
} catch (NumberFormatException e) {
    // ✗ El error desaparece sin dejar rastro
}
```

JAVA - ✗ INCORRECTO

```
try {
    int resultado = Integer.parseInt(texto);
} catch (NumberFormatException e) {
    System.err.println("Formato no válido: '" + texto + "' - " + e.getMessage());
}
```

JAVA - ✔ CORRECTO

Error 2: Capturar Exception genérico demasiado pronto

```
try { leerArchivo(); procesarDatos(); }
catch (Exception e) { System.out.println("Algo falló"); } // ✗ Captura TODO
```

JAVA - ✗ INCORRECTO

```
try { leerArchivo(); procesarDatos(); }
catch (FileNotFoundException e) { System.out.println("Archivo no encontrado"); }
catch (IOException e) { System.out.println("Error de lectura"); }
```

JAVA - ✔ CORRECTO

Error 3: Confundir throw con throws

```
public void metodo() {
    throws new IOException("Error"); // ✗ throws es para la firma
}
```

JAVA - ✗ INCORRECTO

```
public void metodo() throws IOException {
    throw new IOException("Error de E/S"); // ✔ throw lanza, throws declara
}
```

JAVA - ✔ CORRECTO

Error 4: Ordenar catches de general a específico

```
try { leerArchivo(); }
catch (Exception e) { } // ✗ Más general primero
catch (IOException e) { } // ✗ Inalcanzable
```

```
try { leerArchivo(); }
catch (IOException e) { } // ✔ Más específico primero
catch (Exception e) { } // ✔ Más general después
```

Ejemplo completo

```
// Excepción checked: error de negocio
class ProductoNoDisponibleException extends Exception {
    public ProductoNoDisponibleException(String producto) {
        super("Producto agotado: " + producto);
    }
}

// Excepción unchecked: error de programación
class PedidoInvalidoException extends RuntimeException {
    public PedidoInvalidoException(String motivo) {
        super("Pedido inválido: " + motivo);
    }
}

class Inventario {
    private String[] productos = {"Portátil", "Teclado", "Ratón", "Monitor"};
    private int[] stock = {5, 10, 0, 3};

    public void verificarDisponibilidad(String producto, int cantidad)
        throws ProductoNoDisponibleException {
        for (int i = 0; i < productos.length; i++) {
            if (productos[i].equalsIgnoreCase(producto)) {
                if (stock[i] < cantidad) {
                    throw new ProductoNoDisponibleException(
                        producto + " (disponible: " + stock[i]
                        + ", solicitado: " + cantidad + ")");
                }
                stock[i] -= cantidad;
                System.out.println("✓ Reservado: " + cantidad + "x " + producto);
                return;
            }
        }
        throw new ProductoNoDisponibleException(producto + " (no existe en catálogo)");
    }
}
```

```
class Pedido {
    private String cliente;
    private String producto;
    private int cantidad;
    private double precioUnitario;

    public Pedido(String cliente, String producto, int cantidad, double precio) {
        if (cliente == null || cliente.isEmpty()) {
            throw new PedidoInvalidoException("El nombre del cliente no puede estar vacío");
        }
        if (cantidad <= 0) {
            throw new PedidoInvalidoException("La cantidad debe ser mayor que 0");
        }
        this.cliente = cliente;
        this.producto = producto;
        this.cantidad = cantidad;
        this.precioUnitario = precio;
    }

    public String getProducto() { return producto; }
    public int getCantidad() { return cantidad; }
    public double getTotal() { return cantidad * precioUnitario; }

    @Override
    public String toString() {
        return String.format("Pedido[%s, %dx %s, Total: %.2f €]",
            cliente, cantidad, producto, getTotal());
    }
}
```

```
public class SistemaPedidos {
    public static void main(String[] args) {
        Inventario inventario = new Inventario();

        Object[][] datosPedidos = {
            {"Ana", "Teclado", 2, 29.99},
            {"", "Portátil", 1, 899.0}, // Cliente vacío
            {"Carlos", "Ratón", 3, 15.50}, // Sin stock
            {"David", "Monitor", 1, 249.99},
            {"Elena", "Tablet", 1, 199.0}, // No existe
        };

        int procesados = 0, fallidos = 0;

        for (Object[] datos : datosPedidos) {
            System.out.println("\n--- Procesando pedido ---");
            try {
                Pedido pedido = new Pedido(
                    (String) datos[0], (String) datos[1],
                    (int) datos[2], (double) datos[3]);
                System.out.println("Pedido creado: " + pedido);

                inventario.verificarDisponibilidad(
                    pedido.getProducto(), pedido.getCantidad());

                System.out.println("✅ Pedido confirmado: "
                    + String.format("%.2f", pedido.getTotal()) + " €");
                procesados++;
            } catch (PedidoInvalidoException | ProductoNoDisponibleException e) {
                System.out.println("❌ " + e.getMessage());
                fallidos++;
            } finally {
                System.out.println(" [Registro de auditoría actualizado]");
            }
        }

        System.out.println("\n=== RESUMEN ===");
        System.out.println("Procesados: " + procesados);
        System.out.println("Fallidos: " + fallidos);
    }
}
```

```
// Salida:
// --- Procesando pedido ---
// Pedido creado: Pedido[Ana, 2x Teclado, Total: 59.98 €]
// ✓ Reservado: 2x Teclado
// ✓ Pedido confirmado: 59.98 €
// [Registro de auditoría actualizado]
// --- Procesando pedido ---
// ✗ Pedido inválido: El nombre del cliente no puede estar vacío
// [Registro de auditoría actualizado]
// --- Procesando pedido ---
// Pedido creado: Pedido[Carlos, 3x Ratón, Total: 46.50 €]
// ✗ Producto agotado: Ratón (disponible: 0, solicitado: 3)
// [Registro de auditoría actualizado]
// --- Procesando pedido ---
// Pedido creado: Pedido[David, 1x Monitor, Total: 249.99 €]
// ✓ Reservado: 1x Monitor
// ✓ Pedido confirmado: 249.99 €
// [Registro de auditoría actualizado]
// --- Procesando pedido ---
// Pedido creado: Pedido[Elena, 1x Tablet, Total: 199.00 €]
// ✗ Producto agotado: Tablet (no existe en catálogo)
// [Registro de auditoría actualizado]
// === RESUMEN ===
// Procesados: 2
// Fallidos: 3
```

5. EXCEPCIONES IOEXCEPTION CON CLASE SCANNER

Imaginad que estoy haciendo un programa que lee números int del teclado y quiero tratar los errores:

- cuando introducen un char;
- cuando introducen un float;
- cuando introducen un entero más grande que int;

```
public static void Menu()
{
    leer= new Scanner (System.in);
}

public static int lee()
{
    int opciones=0;

    try
    {
        opciones = leer.nextInt();
    }

    Catch (NumberFormatException e) {
        System.err.println("Error");
    }

    return opciones;
}
```

```
    }  
  
    public static void main(String args[])  
    {  
  
        opcion= lee();  
  
    }  
  
    ¡NO FUNCIONA! ¿Qué ocurre?
```

RESPUESTA:

El método `scanner.nextInt ( )` no lanza excepciones, por lo que nunca te saltarán, escribas lo que escribas.

Hay métodos que lanzan excepciones (y en la declaración del método se pone un **throws**). Por ejemplo, el constructor de `FileWriter`, lleva un `throws IOException`. Normalmente, si el método no pone `throws` alguna excepción, no lanzará nada y no tiene sentido poner el `try-catch` (no sirve para nada).

En otras ocasiones pueden saltar excepciones que no están controladas. Los métodos no dicen que las lanzan, pero ocurren normalmente por algún error de programación. Por ejemplo, un `NullPointerException` (Objeto sin instanciar)

En nuestro caso, para resolver esto, en vez de leer un `int` con `scanner`, que no lanza excepciones, intenta leer un `String` y luego convertirlo tú a entero con un `Integer.parseInt()`, que sí lanza excepciones (eso es lo que solemos hacer nosotros).

Código:

```
try {  
    String cadena = leer.nextLine();  
    int opciones = Integer.parseInt(cadena); // Este sí lanza excepciones  
} catch (NumberFormatException e) {  
    ...  
}
```

6. EXCEPCIONES CON SPRING

@ControllerAdvice

`@ControllerAdvice` es una **anotación de Spring** que permite definir lógica **global** que se aplica a todos los controladores (`@Controller` / `@RestController`).

Es como un “**interceptor**” o “**centralizador**” de comportamiento transversal en la capa MVC.

Se usa principalmente para **manejar excepciones de forma global**, pero también tiene otros usos importantes.

Uso principal: Manejo global de excepciones

Sin `@ControllerAdvice`, tendrías que hacer esto en cada controller:

```
try {  
    // lógica  
} catch (Exception e) {  
    return ResponseEntity.status(500).body("Error");  
}
```

Con lo que tendríamos los siguientes problemas:

- Código duplicado
- Difícil de mantener
- Poco profesional

Con `@ControllerAdvice`, lo hacemos una sola vez:

```
@ControllerAdvice  
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(Exception.class)  
    public ResponseEntity<String> manejarError(Exception ex) {  
        return ResponseEntity.status(500).body("Error global");  
    }  
}
```

OJO: Ahora cualquier excepción en cualquier controller pasa por aquí.

¿Cómo funciona internamente?

1. Spring ejecuta un `@Controller`
2. Si ocurre una excepción → Spring la intercepta
3. Busca una clase con `@ControllerAdvice`
4. Busca un método con `@ExceptionHandler` compatible
5. Ejecuta ese método
6. Devuelve la respuesta HTTP personalizada

Componentes clave

`@ControllerAdvice`

Marca la clase como **manejador global**

```
@ControllerAdvice  
public class GlobalExceptionHandler {  
}
```

@ExceptionHandler

Define qué tipo de excepción manejar

```
@ExceptionHandler(UsuarioNoEncontradoException.class)
public ResponseEntity<String> manejar(UsuarioNoEncontradoException ex) {
    return ResponseEntity.status(404).body(ex.getMessage());
}
```

Ejemplo completo (flujo real)

1. Service lanza excepción:

```
throw new UsuarioNoEncontradoException(id);
```

2. @ControllerAdvice la captura:

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UsuarioNoEncontradoException.class)
    public ResponseEntity<String> manejarUsuarioNoEncontrado(UsuarioNoEncontradoException ex) {
        return ResponseEntity.status(404).body(ex.getMessage());
    }
}
```

3. Resultado HTTP:

```
404 NOT FOUND
Usuario con ID 5 no encontrado
```

Otros usos de @ControllerAdvice

Aunque los veréis con más profundidad en la parte de Spring del proyecto final, estas anotaciones no solo sirven para excepciones:

@InitBinder (configuración de datos)

No es específicamente de este tema, pero hay que veces que el @InitBinder se usa en n @ControllerAdvice y permite personalizar cómo se convierten datos (ej: por ejemplo, un String de un formulario a un tipo Date para fechas o String a Enum)

```
@InitBinder
public void initBinder(WebDataBinder binder) {
    // configuración global de conversión
}
```

Versiones especializadas (no son de este curso)

`@RestControllerAdvice`

Es lo mismo que:

`@ControllerAdvice` + `@ResponseBody`

✓ Se usa en APIs REST (más común hoy en día)

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

Ventajas reales

- ✓ Centraliza lógica de errores
- ✓ Reduce duplicación
- ✓ Mejora mantenimiento
- ✓ Permite respuestas HTTP consistentes
- ✓ Facilita logging y auditoría
- ✓ Hace tu API más limpia y profesional

Buenas prácticas

- ◆ Usa excepciones específicas (no solo `Exception`)
- ◆ Define códigos HTTP correctos (404, 400, 500...)
- ◆ No expongas stack traces al cliente
- ◆ Devuelve objetos estructurados (JSON)

Por tanto, `@ControllerAdvice` es como **“un cerebro central que gestiona errores y comportamientos comunes de todos los controladores”**.